

CTF入门指北

CTF?

CTF，即Capture The Flag，起源于黑客之间的游戏，需要对计算机基础有比较深入的了解

简单来说，就是需要在题目给你的附件或者容器环境（比如一个网页）里面，找到或者恢复出一个叫做flag的字符串，通常长这样：flag{This_is_a_flag}，过程中需要用到不限于下面所述的计算机和安全知识与技术

南邮每年10月举办的0xGame也是CTF比赛的一种，不过由于面向新生，内容更偏向计算机基础和安全基础，旨在让新生更好地了解相关知识和技术，0基础通过学习也能有不错的成绩哦！



Web

简而言之的话，Web 基本模拟的是现实世界中网站、API、小程序等基于 HTTP/HTTPS 协议的应用程序。你要做的是通过分析这些应用的代码、逻辑或配置，发现其中存在的安全漏洞，并进行利用。实际体现就是利用各种漏洞一步步getshell或者读取flag。Web上手难度不高，学习资源丰富，网上有很多的博客教程，但是后期想深化会学的很广，考的也会比较难，技术迭代也会比较快。不过我想只要肯花功夫不存在学不懂的问题。

入门知识点：

- 首当其冲是http协议，web世界的根基。
- 然后是前端代码审计能力，从基础的f12查看源码到对于JavaScript等前端语言的基础了解，至少要能看得懂的程度，然后就是对于前端暴露的各种api的挖掘能力。
- 后端代审能力也是不可或缺的，首先最该学的是python，不仅在各种python环境下要做代码审计，平常经常会要编写各种解题脚本，也基本是用python实现最方便快捷。然后是各大后端语言如php，java，golang。不要求太强的写代码能力（那是后端开发要学的），但是至少要会认，能写基础脚本。
- Linux基础同样重要，设计到getshell（拿到执行命令点）后获取战果的环节，如用Linux查看各种文件等。
- 然后就是我们碰到的各种漏洞了：
 - **SSTI**：攻击者将恶意代码作为数据注入到服务端模板引擎中，导致模板引擎将其当作代码执行的安全漏洞。
 - **XSS**：攻击者将恶意 JavaScript 代码注入到网页中，其他用户（或 CTF 中的 Bot）浏览该页面时，浏览器执行这段恶意代码
 - **反序列化**：又可以细分为java反序列化和php反序列化，原理是将字节流/字符串还原为内存中对象的过程。当应用程序对不可信数据执行反序列化，且语言/框架的对象重建机制可被操纵时，攻击者就能通过精心构造的序列化数据，在对象创建过程中触发任意代码执行。
 - **身份验证缺陷**：服务器通过jwt，cookie等方式识别用户身份，如果这个认证机制存在缺陷，你可以通过这个来伪装成其他用户甚至直接成为admin（高权限管理者）。
 - **文件上传以及木马**：这两个常绑定着考，就是大家认知里的上传木马，虽然其实有更高级的不死马，内存马等等，但是这里不展开，感兴趣可以了解一下。
 - **沙箱逃逸**：攻击者突破受限执行环境的安全边界，从“只能做有限操作”的状态提升到“拥有宿主完整权限”的过程。比如我们的pyjail。
 - **各种RCE绕过方法**：也就是我们的防火墙绕过，即在已存在命令执行点但受到防御限制的情况下，通过编码、拼接、替代函数、协议滥用等手段，使恶意 payload 成功执行的技术集合。这个要慢慢积累。
 - **文件包含**：应用程序将用户可控输入作为文件路径参数，动态加载并执行/读取目标文件的安全漏洞。当开发者使用 `include`，`require`，`file_get_contents` 等函数时未对路径做严格校验，攻击者就能让服务器“包含”任意本地或远程文件。
 - **SQL注入**：攻击者将恶意 SQL 代码片段插入到应用程序的输入中，使后端数据库将其误认为合法查询指令并执行的安全漏洞。比如我们的“万能密码”。
 - **SSRF**：攻击者构造恶意请求，诱导服务器端向攻击者指定的内部或外部地址发起 HTTP/其他协议请求的安全漏洞。常用来访问内网资源，或者加载恶意代码。
 - **XXE**：当应用程序解析 XML 输入时，若未禁用外部实体引用，可通过构造恶意 DTD/实体声明，诱导 XML 解析器发起任意文件读取、SSRF、命令执行等攻击的漏洞。

- **原型链污染**：通过操纵对象的 `proto`、`constructor.prototype` 或 `Object.prototype` 属性，向所有对象的原型链中注入恶意属性或方法，从而改变程序全局行为的漏洞。（其实除了js其他语言也有类似的东西）。
- 再然后就是对于各种自动化工具的使用了，比如typhon（<https://github.com/LamentXU123/Typhon>），fenjing（<https://github.com/Marven11/FenJing>）等，也是要自己慢慢积累，不过使用这种自动化一把梭的前提最好是自己先懂得原理，看的懂最后用的payload它为什么这么构造。

如何学习：

在现在ai时代之下，善用ai是必备的技能，毕竟有哪个老师有ai知识面覆盖的广，又有它孜孜不倦呢？在前期代码基础没那么厚实的情况下你可以让ai给你逐段讲解需要审计的代码，也可以让它讲讲这个payload为什么这么构造，原理是什么。

然后就是善用搜索引擎，去网上找相关讲解的博客也是必不可少的一环，要善用互联网资源，你可以在上面找到各种专精讲解，以及对某个漏洞的POC（就是利用代码）等。

以及要学会对自己学的东西进行总结归纳，Web设计到的知识很多很杂，我个人的方法就是对所有学到的东西都梳理成笔记，防止自己忘记（而且有时可以直接喂给ai让它做题）。

最后可以多参加一些比赛，比如各种新生赛等，边学边练是最好的提升方式，而且可以接触很多比较新的知识点，利用方法。

基础工具配置：

Web相对其他方向入门环境配置应该比较简单，最重要的就是一个抓包工具。跟着这个文档来就行 [📖 CTF常用工具安装](#)。

一些资源：

- <https://xz.aliyun.com/news>
- <https://www.cnblogs.com/>
- <https://www.ctfplus.cn/>
- <https://platform.cyclens.tech/>
- <https://platform.cyclens.tech/>

Misc

比较好入门，但是深入学习会发现内容很多的一个方向

| 俗话说的好，不知道放哪个方向就放misc里面来

如果按照传统的五大方向分，misc会掺杂很多本不属于misc的内容：ai，blockchain，数据处理等等

这里只讲传统的misc知识

misc入门知识点

传统的misc有信息搜集，编码学，隐写术，流量分析，取证五个方向

- **信息搜集（Osint）**：这是全方向都需要的东西。首先，使用bing/Google搜索引擎。其次，学会搜索关键词。比如，搜索一个长的报错，需要抓住报错的特征。
 - **地理定位（图寻）**：给出一张照片（可能是从窗外拍的、火车站、街景），通过图片中的地标、特定文字、电线杆类型、影子角度，通过 Google Map/百度地图等定位拍摄者精准的经纬度。
 - 图片的exif元信息有时会藏着经纬度
 - Google/百度识图是识别常见地点很好用的一招
 - 豆包的图寻能力也未尝不可使用 实测很强👍
 - **社会工程学检索**：根据已知人物的名字、曾用网名，去各大平台（GitHub、微博、贴吧、Bilibili、CSDN等）检索其历史痕迹。
 - **历史快照检索**：利用 Wayback Machine（网页时光机），如[web archive](#)，寻找已被删除的网页历史记录或 GitHub 被删除的 Release/Commit 历史。
 - **列车/航班/船舶追踪**：通过照片里出现的飞机编号或船舶名称，搜索专门的信息查询网站，查询当时拍摄者所在的航线和位置。
 - 为了达到一个目的，以上的方法是可以交叉使用的，进行双重/多重验证，确保信息准确
- **编码学**：日常可以见到的非常多，如base64，hex等传输编码，摩斯密码等古老通信密码；同时也有古典密码，如凯撒，维吉尼亚等等，这些通常只需要会使用工具就可以解出，常常和其他知识点一起考察，是所有方向的基础
 - 二维码也是一种编码：
 - 二维码本身藏有信息
 - 根据残缺的二维码恢复二维码数据（注意，获取二维码内数据并不需要完整二维码）
 - 将已有的数据转化为二维码（二维码就是0和1，黑和白的序列）
 - 如果有隐写的参考图像隐写部分
 - 一些奇怪的特定情境编码：善于搜索
 - 推荐随波逐流/Cyberchef自动识别
- **隐写术**：首先要了解不同文件的特征，会用010和binwalk，foremost等工具查看和分离直接的隐藏数据/文件，以及一些隐写工具的反向解密
 - a. **图像隐写**：lsb（最低有效位），exif元数据，steg（RGB位图片藏东西），盲水印（傅里叶变换），gif帧隐写，宽高被修改，以及一些图像的加密和混淆；silent eye，deepsound，oursecret等专用工具隐写

- b. **音频隐写**：一般用Audacity，查看音频的频谱和波形图；如果藏了二进制信息的，用python写脚本分析解码；比较简单，特征明显的SSTV
- c. **文本隐写**：零宽隐写，snow隐写等等
- d. docx/doc/pptx隐写：这些的本质都是压缩包，解压可以查看隐藏的结构
- e. git/docker隐写/取证
- f. **zip压缩包**：也算在隐写里面，是对zip的一些破解手段
 - i. zip伪加密：看上去加密实则没有，部分工具可以直接打开
 - ii. 弱密钥：你的密码太弱了！archpr/在线网站爆破
 - iii. 已知明文攻击：特定存储方式和存储文件的zip可以通过bkcrack通过破解
 - iv. crc爆破：短文本的crc爆破成本很低
 - v. 当然，大部分zip是安全的，必须从其他地方获取密码
- **流量分析**：对计算机网络中的流量包（.pcap/.pcapng文件）进行分析。了解流量中进行了什么操作，传输了哪些内容，有时结合隐写术考察；有时也作为取证，对攻击者行为进行分析。我们要过滤出我们需要的流量进行分析，有时这部分流量是加密的，还需要获得密钥
 - 一般使用wireshark进行分析
 - 需要对**计算机网络**知识比较熟悉，了解各种流量的类型和攻击手段
- **取证**：一般分为内存取证和磁盘取证
 - **内存取证**：提取计算机RAM的**实时进程**作为内存镜像进行分析，捕捉瞬间的快照。一般使用volatility2或者3等进行分析
 - **磁盘取证**：提取计算机的整个磁盘镜像，可以看整个磁盘的文件，恢复被删除的数据等等。一般使用FTK，Autopsy等工具进行分析
- 其他：
 - Linux操作，基本命令和对应参数（id,whoami,ls,cat,grep……），通配符，渗透方向
 - pyjail沙箱，需要掌握python语法和一些特性
 - 游戏（玩通关或者CE改条件），如果用逆向方法做就涉及到一些Revese的知识
 -

同时，你需要会一点其他各方向的基础，在面对较为复杂和新颖的misc题时不至于卡住

工具配置

很多题目可以用搜索引擎搜到很多在线的web工具，但是有时也需要本地工具的配置。

关于本地工具的下载和使用，建议善用搜索引擎，不要下载到盗版和投毒的（很危险！）

实在不行也可以单独找misc方向的师傅们要（Spring的QQ:2922562941）

agent使用

对于新手，misc方向大多数时候无需Agent。对于解题的python脚本，简单的建议手搓，遇到一些困难的也可以求助ai；纯自动化做题，对于基础薄弱的新手是不可取的。

后续遇到一些困难的题目可以考虑使用

建议采用Chat模型问答+手动搜索/使用工具做题的方式进行

我们在0xGame中的题目也不会出很难的脚本分析题，为的就是鼓励大家手做

刷题平台

除了NSSCTF、BUUCTF、CTFHub、攻防世界这些，还推荐：

- 直接参加比赛：以赛促学，misc就是要见识广
- [xdu的平台](#)，有很多复现，moectf也是不错的入坑比赛
- [ctfplus](#)，去年和今年0xGame的合作伙伴，有0xGame2025的复现环境
- 一些其他的新兴靶场，我知道的比如polar，青岑，好靶场等等

为了奖励你的认真观看，这里有一串神秘字符送给你：

MHhHYW1le3dlbGNvbWVfdDBfdGhlX3cwcmxkX29mX0NURiF9

这是什么呢？学会使用搜索引擎和AI吧！

Reverse

reverse简介

简单来说，逆向工程（Reverse Engineering）是一个拆解程序理解逻辑的过程：把已经做好的软件/程序拆开，分析它的内部结构、运行逻辑和算法。

比如：

分析游戏安装包，自己写脚本开挂（~~bushi~~）咳咳，省去重复操作提升游戏体验，或者自己加装mod
破解加密文件，找出隐藏在背后的密码规则，还原加密逻辑

而在ctf竞赛的reverse方向中，通常来说我们会获得一个可执行文件。以 linux 系统下的逆向为例，我们会获得一个 elf（Executable and Linkable Format）文件。

一般这个文件是可以直接运行的（Windows 系统下可以使用虚拟机），但是直接运行往往不能拿到我们想要的东西。

这时候我们就需要工具，来分析这个文件了。通常来说针对不同的文件，我们要用的工具也不一样。一个比较通用并且好用的工具就是著名的 IDA Pro（简称IDA），至于获得方式，你都学逆向了，还要问我正版软件买不起怎么办？我是不会告诉你的！善用搜索（笑）

通过你能想到的所有办法分析，既是逆向工程的难点，也是逆向工程的乐趣所在。那种山重水复疑无路，柳暗花明又一村的体验，希望你也能体会到。

入门学习

1. **各种编程语言。**首先应该学会的就是C语言，作为一门相当底层的高级语言，学会C语言的好处多多，此处省略100字。当然你硬是要学 Python/Java/Kotlin/C#/Go/Rust/... 也不是不行，总之多多益善。但要注意我们学习语言并不是学会语法就行了，了解一种语言运行的基本逻辑，才对我们的逆向有所帮助。
2. **基本的加密算法。**如果 flag 永远只是一个躺在程序里的明文，那你用的最多的工具应该是 010 editor (笑)。对输入加密是基本操作，包括但不限于 xor/AES/tea/xtea/xxtea/rc4等等对称加密，和各种你见过的没见过的非对称加密。能看出一段代码是哪种加密，你离成功就更进了一步。
3. **静态分析。**阅读底层语言或许很难，但是好在我们有 IDA 的反汇编功能，只需要简简单单按下 F5，我们就能获得类似阅读源码般的体验。当然，事情不会这么简单，防止软件被静态分析的方法很多，例如 花指令/upx 加壳/SMC/ollvm 等等，而对应的绕过方法也是存在的。而模拟执行、符号执行等方法也可以使程序的逻辑变得清晰，不过技术难度也更大。
4. **动态调试。**软件一定要可以运行！既然可以运行，我们就有各种方法通过调试获得内存里的东西。例如通过 Cheat Engine 搜索内存，或是用 IDA/x64dbg 等软件跟踪运行过程，找到执行逻辑。而反调试技术也随之出现，想要防止我们知道程序实际执行起来是什么样子，例如 TLS callback/SEH 异常处理 等等。

同时，你需要会一点其他方向的基础：比如 misc 的编码和流量、pwn 的栈和内存、crypto 的常见算法，不然遇到稍微综合一点的 reverse 题会很容易卡住。

工具配置

很多 reverse 工具都需要自己慢慢配环境，尤其是不同平台差异很大。

关于工具的下载和使用，建议善用搜索，我这里放下一些常见的。

IDA

<https://www.52pojie.cn/forum.php?mod=viewthread&tid=2117154&highlight=ida%2B9.4>

x96dbg(x64dbg+x32dbg)

<https://x64dbg.com/>

010 Editor

https://down.52pojie.cn/Tools/Editors/010_Editor_All_Versions_For_Windows_Cracked.7z

upx

<https://github.com/upx/upx/releases/>

dnSpy

<https://down.52pojie.cn/Tools/NET/dnSpy.zip>

Die

<https://www.52pojie.cn/thread-1736240-1-1.html>

agent使用

对于新生，reverse 方向不建议一开始就依赖 Agent。

因为 reverse 的核心能力就是自己建立“程序到底在干什么”的理解，如果一上来就让 ai 帮你解释，很容易看懂答案但不会自己做。

比较推荐先自己看，自己调，有具体卡住的地方再询问ai

让 ai 帮你总结函数逻辑、解释调用约定、写还原脚本，这些都很好，但不要把整个题一股脑丢给 ai 自动化做完。

刷题平台

misc师傅说的那些平台都很好，我这里再补充一下，还可以去一些技术交流论坛上学习其他师傅的思路，也有很多资源分享，这里推荐几个。

看雪，吾爱破解，ctf-wiki也可以看。

Pwn

pwn简介

在CTF竞赛中，Pwn是最具挑战与魅力的领域之一，也是通往二进制安全研究的必经之路。Pwn题目通常会给你一个运行在远程服务器上、存在漏洞的二进制程序。你的任务就是分析这个程序，编写利用脚本，通过网络与之交互，触发漏洞，最终在服务器上获得一个Shell，读取到隐藏的Flag。但是往往有的时候出题人会在这方面挖坑，如果getshell的文件与flag文件不在同一目录下该怎么办，如果getshell了，但是你所有的输入都得不到输出怎么办？...

环境配置

可以看下博客来配置环境：

<https://xi->

[zi.github.io/2026/07/05/Pwn%E5%85%A5%E9%97%A8%E7%8E%AF%E5%A2%83%E6%90%AD%E5%BB%BA/](https://xi-zh.github.io/2026/07/05/Pwn%E5%85%A5%E9%97%A8%E7%8E%AF%E5%A2%83%E6%90%AD%E5%BB%BA/)

当然如果在安装环境的时候出现了问题，也欢迎在群里的小窗点戏子师傅。(QQ:1561898724)

常用查询libc版本网址：

<https://libc.rip/>

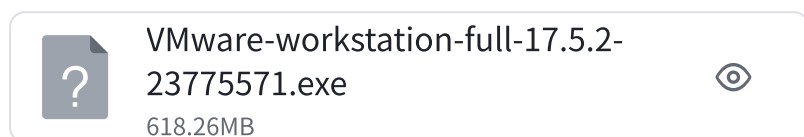
<https://libc.blukat.me/>(这个好像崩了？)

前期学习

在正式开始Pwn学习之前，需要具备以下基础：

1. 学懂C语言。C语言是信息类的基础，Pwn题目绝大部分用C/C++编写。当然C++其实也是少数，前期有C语言的基础就够用了，至少要明白各个函数究竟在干什么，配合ida工具的使用，会让你对pwn附件的了解事半功倍。前人栽树。后人乘凉啊。工具的获取参考这个链接：[CTF常用工具安装](#)

上面基础工具的安装好像崩了。所以这里给出vmware workstation的安装包。自取



2. 基本使用Ubuntu系统：掌握 `sudo`、`apt`、`ls`、`cd` 等基本命令。其实没有特别大的要求，你只需要知道当你getshell以后，怎么能读到flag就行了（如果你能找到的话hh
3. 理解栈的生成与销毁。这些内容都可以上b站大学去了解，也有很多优秀的师傅分享了很多通俗易懂的方法。这个对你后续的学习起到了相当大的作用，你只有理解底层程序是如何运行的，你才能劫持程序的执行流。
4. 计算机体系结构基础：了解内存、地址、寄存器等概念。这里其实只要关键是对寄存器的了解。知道函数是如何调用的，参数和寄存器之间的关系是什么，想必了解了这些的你，在面对学校的C语言课，就可以直接起飞了（bushi

推荐学习顺序：

1. ret2text：最简单的栈溢出，通过覆盖返回地址跳转到后门函数（如 `system("/bin/sh")`）
2. ret2shellcode：在栈上布置shellcode并跳转执行，了解什么是shellcode？怎么手编shellcode，shellcode中什么是坏字符？
3. ret2libc：利用libc函数绕过NX保护，怎么调用程序中不存在的函数？

后续还有ret2rop, ret2csu, srop, brop, FMT等等十分经典的手法，一定会让你眼前一亮的。🔧

学习任务：

- 学会使用 `checksec` 查看程序保护，有的时候可能看到保护就知道怎么做了🤖
- 掌握pwntools的基本语句：`p64()`、`p32()`、`process()`、`recv()`、`sendline()`、`interactive()`，这些都是你脚本的组成非常重要的成员哦。千万不要让ai帮你写脚本，因为你根本看不懂，这没有益处，一定要试着自己手搓脚本，这样会有不一样的体验哦。
- 理解栈帧中 `rbp` 和返回地址的位置关系，这就是我们劫持程序执行流的关键哦，不过好像和异架构不同，后续有兴趣可以了解😓

等对栈利用有了认识以后，可以慢慢对堆，IO等方向进一步的学习，当然这都是后话。

如果在刚开始学习的过程中感到了困难，是很正常的，欢迎看戏子师傅的博客来解决问题，如果解决不了或者觉得不直观，也可以来问戏子师傅。当然，在问问题之前，希望你仔细阅读了西电平台的《提问的智慧》。

<https://ctf.xidian.edu.cn/wiki/7>

后期深化

- libc源码阅读：理解glibc的堆管理实现
- 汇编语言精进：能够熟练阅读和编写x86/x64汇编
- ARM架构Pwn：使用QEMU模拟（这个方向和IoT很相关，也是异架构汇编的一种

下面三个方向选一个学习

- 内核Pwn：学习内核驱动中的漏洞和提权技术（如果不读研，不建议研究
- 浏览器Pwn：V8引擎漏洞利用（国内不太建议学习...
- IoT设备漏洞挖掘（如果想实习，可以选择

持续学习建议

- 阅读高质量的漏洞分析报告和Writeup
- 参加CTF比赛，在实战中锻炼（在赛后复现吧...
- 关注安全社区（如FreeBuf、安全客等）的最新漏洞披露
- 建议在学习初期先搭建自己的博客，不一定非要买服务器，用github代理就可以的，平常纪录自己眼前一亮的题，或者是各个比赛的wp，对pwn的学习进程，各个手法的利用等等。保持长期学习的好习惯👉
- 可以向各大战队投递简历，这对后期的发展也是十分有帮助的。比如：星盟安全团队，V&N，SU等等

agent的使用

在入门阶段，你可以将AI作为学习助手：

1. 代码理解：将反编译的伪代码喂给AI，帮助理解程序逻辑。不过在前期的学习中，建议对于哪些内容很短的伪代码还是自己解读，提升对于漏洞的敏感性
2. 概念解释：遇到不理解的概念（如ret2libc、GOT劫持等），向AI提问，底层的问题问AI，绝对是一个明智之举
3. 调试协助：描述调试中遇到的问题，获取排查建议

注意事项

- AI生成的代码需要人工验证，不可直接用于比赛
- 理解原理比依赖工具更重要——先用手动方式完成，再用工具自动化
- 在CTF比赛中，使用自动化Agent需遵守比赛规则
- 希望不要成为豪哥，只会用agent而没有自己的理解，缺少对于底层的探索，终究不会让你走的很远

刷题平台看看misc师傅推荐的就行了。

最后我想说，pwn的入门比较吃基础，一开始在pwn的世界里遇到挫折很正常。同时，也需要我们多加坚持，多付出努力。相比较于现实的世界，二进制的世界不会说谎。希望你能在学习pwn的过程中得到提升和快乐。我希望与你在pwn的世界相遇，一起努力，一起进步。

Crypto

Crypto概览

作为信息安全的基础，适配最完美的体系与不错的未来扩展前景，密码学同时也作为学校的课程，但是CTF密码学不同，学校课程为学习防御密码体系是如何构造，CTF密码学是标准防御密码体系，在特定情况下会呈现出数学漏洞、结构漏洞，以至于可以获取私钥、篡改签名等，入手难度为中等，后续也可以比较合适的转到通信安全、车联网等方向，需要数学分析能力、模型构建能力、记忆力，以及不错的Python编程能力

环境配置

密码作为现有工具最少的方向，大部分脚本都是根据经验预制或者现场写，编程大部分使用Python与sagemath,目前主流是搭配agent实现人与AI互助

Python配置这里不展开，以下是一些可用的sagemath部署命令,在powershell中运行，预期时间从20分钟到三个小时不等（仅供参考）,预计要10G左右内存，下面示例使用的是Ubuntu虚拟机，清华大学的镜像

代码块

```
1 wsl -- bash -lc "sudo apt install -y m4 git python3 python3-dev sqlite3 bc  
cmake curl libcurl4-openssl-dev pkg-config autoconf automake libtool liblzma-  
dev libgmp-dev libntl-dev"  
2 wsl -- bash -lc "gcc --version && g++ --version && make --version && python3 --  
version && git --version"  
3 wsl -- bash -lc "cd /path/to/sage && ./configure"  
4 wsl -- bash -lc "cd /path/to/sage && ./configure"  
5 wsl -- bash -lc "cd /mnt/c/Users/Lenovo/Desktop/sage && ./configure"
```

前期学习指南

先是最基础的Python学习，要求能够会使用所有的最基础语法，会写水仙花数为最基本的程序能力要求，后续因为不同库而使用不同语法特征，后续遇见一次就学习新的知识，sagemath并非早期编程能力，本质Python的扩展，可以学习到ECC相关可以再开始了解

对于最早期的密码知识学习，先学习最基础的古典密码系列，如凯撒密码、维吉尼亚密码，要求能够自己手搓程序，并且了解如栅栏密码等其他古典密码，在同时了解哈希函数的基础概念，一些经典加密模块，比如说AES模式ECB等的基础概念，要能够学会相关基础代码读写，对基础编码知识，比如base64，要有原理上的理解。对哈希函数的程序能力要求能够自然地利用哈希函数，以及对哈希函数的缺位爆破，具体可以参考0Xgame2025week1的题目。接下来是进入RSA系列的学习,先了解基础概

念,以及模运算的基本,了解快速幂算法等模的基础运算,具体可以参考下面的师傅的博客中RSA部分,接下来是ECC部分,在这个阶段就会接触到sagemath语法,学习常见的ECC漏洞,之后在学习分组密码模式的常见漏洞,接下来就是伪随机数与格密码部分,完成以上基础学习之后就可以跟随比赛学习更进阶的题目。

可供参考的链接

<https://seandictionary.top/>

<https://lazzaro.github.io/categories/crypto/>

后期能力转型进化

可以专精一部分的知识,比如格密码,学习和密码属性相关知识,将开发的知识套用到其他产品上,实现创新,以便于打作品赛。根据已有签名知识,协议知识,公钥密码的基础,可以较为轻松的转同类型比赛的通信安全,车联网相关赛道。

或者联系相关老师,转型密码开发相关

agent使用

密码赛道是受AI智能冲击最大的赛道之一,个人并不排斥对AI脚本的使用,因为AI编写脚本的速度远超前于师傅,但是对于AI脚本的编程知识,要谨慎选择学习,对于陌生的题目可以交给AI进行知识点鉴定,进行学习,但不能AI打完之后就草草略过,在有了基础之后,自己参与agent开发配置知识库,实现当下人机配置

首次使用可以使用GPT的桌面应用codex

刷题平台

不推荐其他方向推荐的 buu等平台,其中有太多古典密码滥竽充数以及将一些冷门怪的知识点作为难题,推荐去年0x week1至3的知识(在ctf plus平台)

如果后续还有刷题要求,可以直接联系我本人,QQ 3431226260